Class 9

Tuesday, June 23, 2026 9:08 PM

Amazon

 → placing order → payment

↓

→ Credit Card
→ Debit Card
→ Wallet

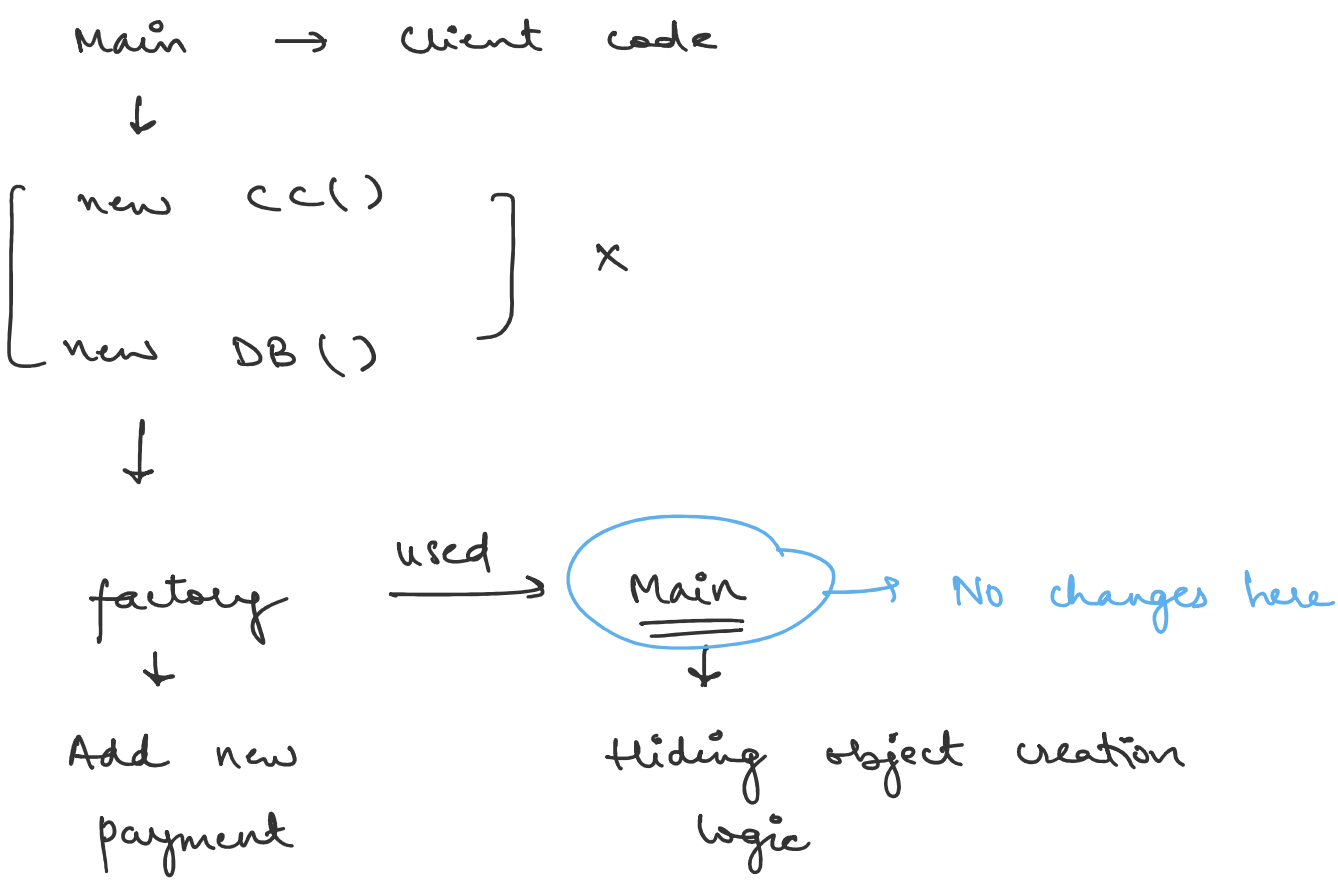
factory Design pattern

factory — creates products of same type

Payment — type

↓

creating objects of payment type


Main → Client code
new CC()
new DB()
x
factory
used
Main
No changes here
Add new payment
Hiding object creation logic


Client
use
factory
create
Products

Real life eg –

→ order food ✓

↓

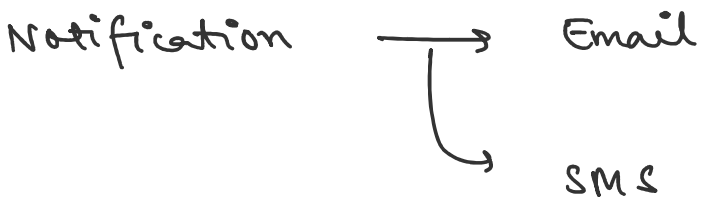
Get it

- → What temp used for baking
- → What ing they must have put
- → How it is prepared.

→ Real factory layout

- → Category
  - ↳ Interface
- → Actual Product
  - ↳ child classes
- → Establish factory
  - ↳ factory class.

Another eg –



Steps –

1. Create enum for Notification Type
2. Create interface Notification
3. Create concrete classes – Email Notification, SMS Notification.
4. Create factory class – Notification factory

public static Notification getNotify

(NotifType type)

{

_____

_____

_____

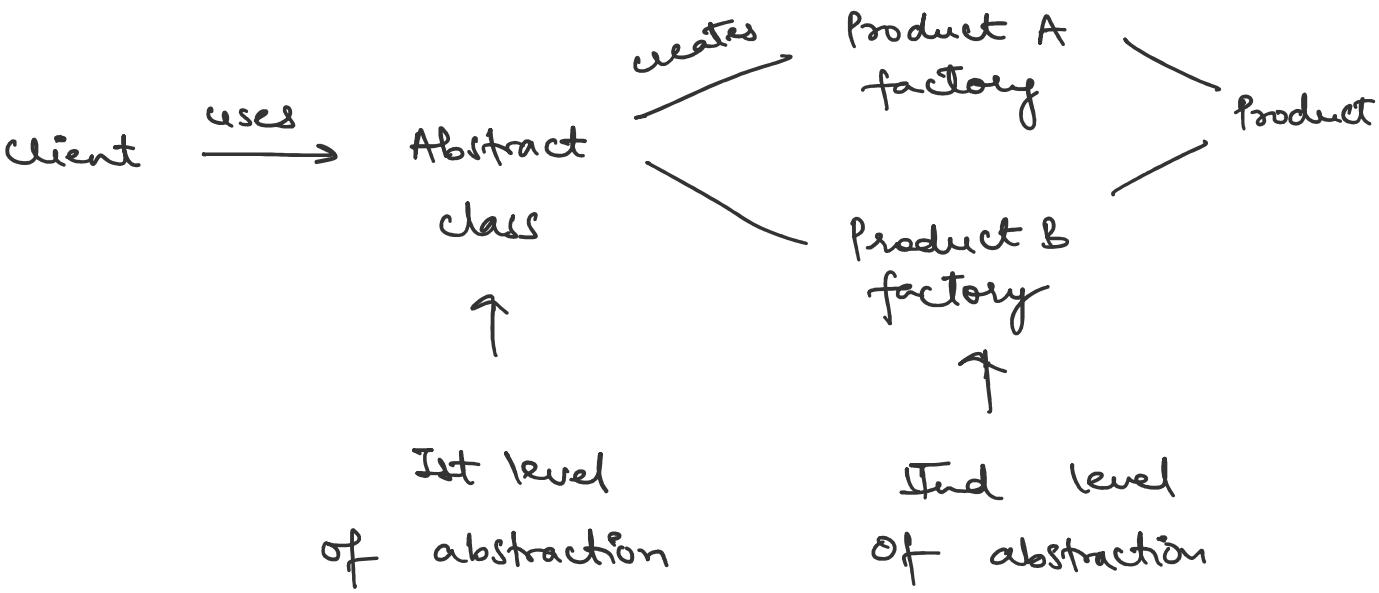
}

5. Create client code – Main method.

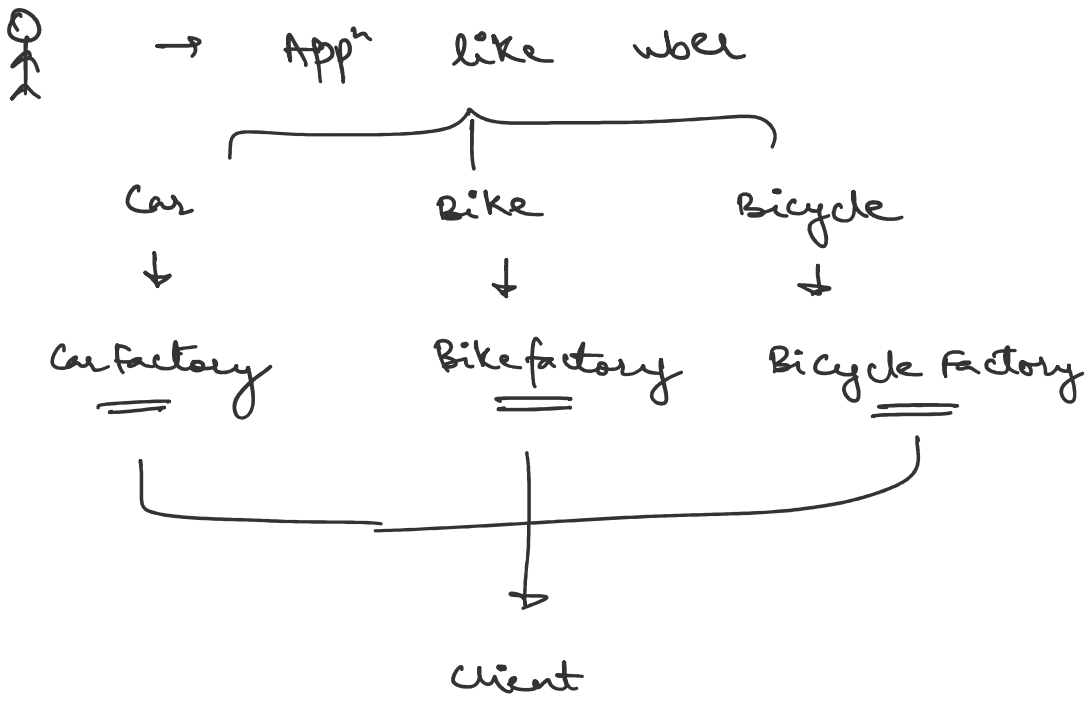
Simple factory

client  factory  Products

factory method → 2 level of abstraction


Client
uses
Abstract class
creates
Product A factory
Product B factory
Product
1st level of abstraction
IInd level of abstraction

Real life eg -


App^n like ubel
Car
Bike
Bicycle
Car factory
Bike factory
Bicycle Factory
Client

Factory design pattern is something which will come to rescue when we have to perform mass production of similar kind of objects. When using the **Factory Pattern**, think of a **real-world factory -** A factory produces **different versions** of a product based on a request.

Factory Pattern is a **Creational Design Pattern** that encapsulates object creation logic inside a factory class. It is used when you **don't want to create objects directly using new**, instead, you let a **Factory class** decide and create the object based on some input.

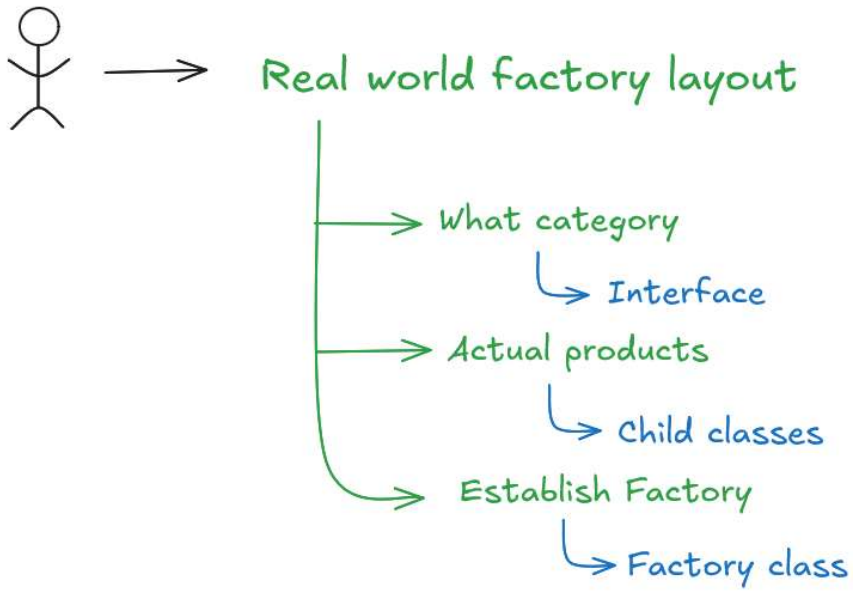
It **hides the object creation logic** from the client, the client requests the factory to create the object. This makes the system **flexible** and **easy to extend**.

Real-Life Analogy – Ordering Food

Think of it like ordering food, you tell the **waiter** what you want. The **kitchen** decides how to make it. You just get the food – you **don't care what ingredients to put into it, how it was prepared or what is the oven temperature on which it was cooked**.

Same with factory pattern:

- You ask the **factory** for an object.
- It creates the object and gives it to you.
- You never directly use new.



Problem Without Factory Pattern

Suppose we have multiple payment methods.

```
if(paymentType.equals("UPI")) {
    payment = new UpiPayment();
}
else if(paymentType.equals("CARD")) {
    payment = new CardPayment();
}
else if(paymentType.equals("NETBANKING")) {
    payment = new NetBankingPayment();
}
```

Problems:

1. Client knows all implementations.
2. Every new payment type requires modifying client code.
3. Violates Open Closed Principle.
4. Object creation logic gets scattered throughout the application.

Solution

Move creation logic into a factory.

```
Payment payment =
        PaymentFactory.getPayment(paymentType);

payment.processPayment();
```

Now client doesn't know:

```
new UpiPayment()
new CardPayment()
new NetBankingPayment()
```

Factory handles everything.

Example 1: Payment Gateway

Requirement

Support:

- Razorpay
- Stripe
- PayPal

Used In

E-commerce

- Amazon
- Flipkart
- Swiggy
- Zomato

Without Factory

```
if(type.equals("RAZORPAY")) {
    return new RazorPayGateway();
}
```

Everywhere in code. Bad design.

With Factory

1. Define Enum for payment

```
public enum PaymentType {
    CreditCard,
    DebitCard
}
```

2. Define an interface Payment (common type)

```
public interface Payment {
    void pay(PaymentType type);
}
```

3. Create child classes that implement the interface.

(i) CreditCard

```
public class CreditCard implements Payment {
    @Override
    public void pay(PaymentType type) {
        System.out.println("Paying by " + type);
    }
}
```

(ii) DebitCard

```
public class DebitCard implements Payment {
    @Override
    public void pay(PaymentType type) {
        System.out.println("Paying by " + type);
    }
}
```

4. Write a factory class PaymentFactory that returns the correct object based on input.

```
public class PaymentFactory {
```

```
    public static Payment getPayment(PaymentType
paymentType) {
        switch (paymentType) {
            case CreditCard:
                return new CreditCard();
            case DebitCard:
                return new DebitCard();
            default:
                throw new IllegalArgumentException("Invalid payment
type: " + paymentType);
        }
    }
}
```

5. Client code FactoryPattern will only call the factory, not use new.

```
public class Main {
    public static void main(String[] args) {
        Payment payment1 =
PaymentFactory.getPayment(PaymentType.DebitCard);
        payment1.pay(PaymentType.DebitCard);  // Output: Paying
by Debit Card

        Payment payment2 =
PaymentFactory.getPayment(PaymentType.CreditCard);
        payment2.pay(PaymentType.CreditCard);  // Output: Paying
by Debit Card
    }
}
```

Example 2: Credit Card Service

Requirement
Get credit card details (card type, card limit, card annual fee) for card types:

- Platinum
- Titanium
- Gold

1. Define an enum

```
public enum CreditCardType {
    PlatinumCard,
    TitaniumCard,
    GoldCard
}
```

2. Define an interface (common type).

```
public interface CreditCard {
    String getCardType();

    int getCreditLimit();

    int getAnnualCharge();
}
```

3. Create child classes PlatinumCard and TitaniumCard that implement the interface.

(i) PlatinumCreditCard

```
public class PlatinumCreditCard implements CreditCard {
```

```
    @Override
    public String getCardType() {
        return CreditCardType.PlatinumCard.toString();
    }

    @Override
    public int getCreditLimit() {
        return 100000;
    }

    @Override
    public int getAnnualCharge() {
        return 100;
    }
}
```

(ii) TitaniumCreditCard

```
public class TitaniumCreditCard implements CreditCard {
    @Override
    public String getCardType() {
        return CreditCardType.TitaniumCard.toString();
    }

    @Override
    public int getCreditLimit() {
        return 500000;
    }

    @Override
    public int getAnnualCharge() {
        return 500;
    }
}
```

(iii) GoldCreditCard

```
public class GoldCreditCard implements CreditCard {
    @Override
    public String getCardType() {
        return CreditCardType.GoldCard.toString();
    }

    @Override
    public int getCreditLimit() {
        return 1500000;
    }

    @Override
    public int getAnnualCharge() {
        return 1500;
    }
}
```

4. Write a factory class CreditCardFactory that returns the correct object based on input.

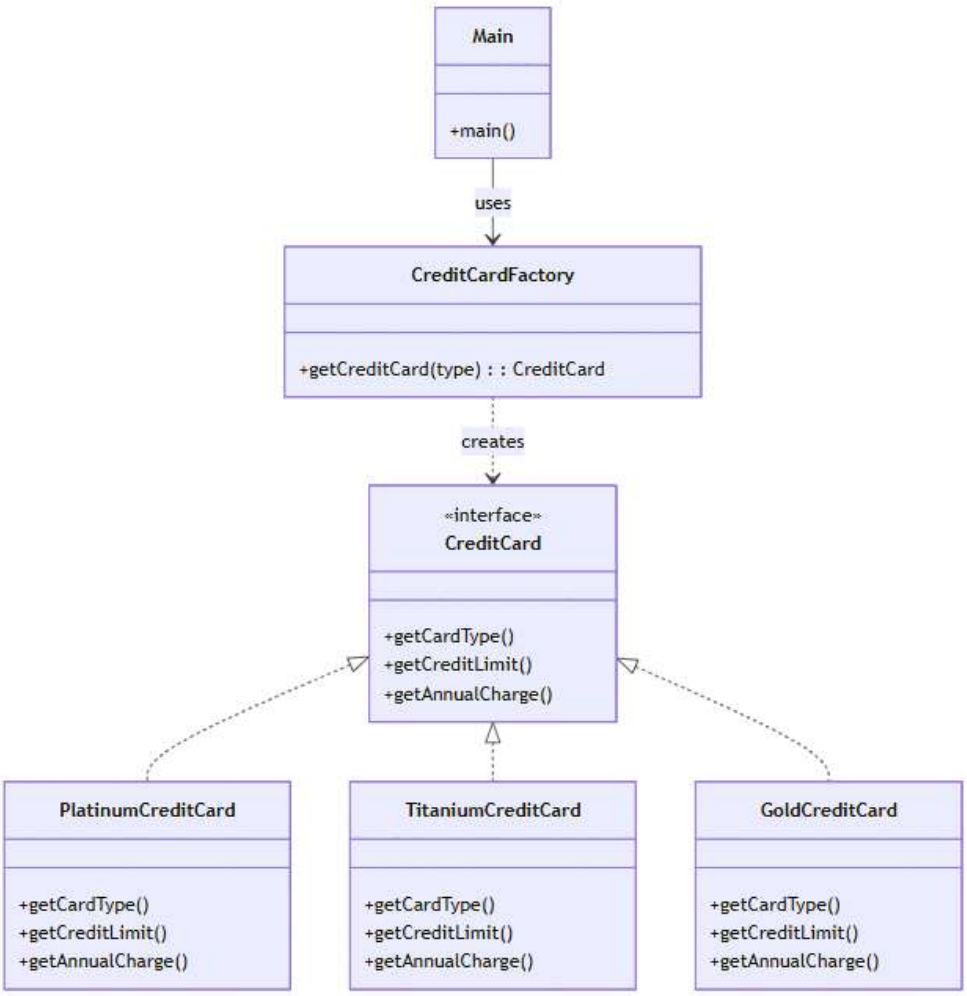
```
public class CreditCardFactory {
    public static CreditCard getCreditCard(CreditCardType type) {
        switch (type) {
            case PlatinumCard:
                return new PlatinumCreditCard();
            case TitaniumCard:
                return new TitaniumCreditCard();
            case GoldCard:
                return new GoldCreditCard();
            default:
                throw new IllegalArgumentException("Invalid credit
card type: " + type);
        }
```

```
    }
}
```

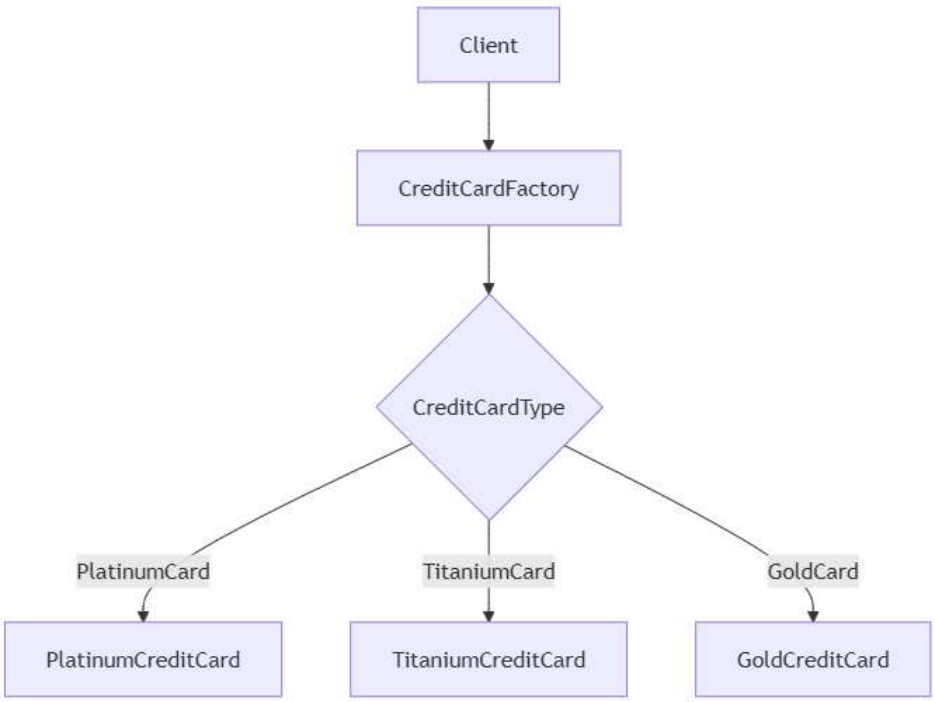
5. Client code will only call the factory, not use new.

```
public class Main {
    public static void main(String[] args) {
        for (CreditCardType type : CreditCardType.values()) {
            printCardDetails(type);
            System.out.println();
        }
    }

    private static void printCardDetails(CreditCardType type) {
        CreditCard card = CreditCardFactory.getCreditCard(type);
        System.out.println("Card Type      : " + card.getCardType());
        System.out.println("Credit Limit   : " + card.getCreditLimit());
        System.out.println("Annual Charge  : " +
card.getAnnualCharge());
    }
}


/* *
    Output :
        Card Type      : PlatinumCard
        Credit Limit   : 100000
        Annual Charge  : 100

        Card Type      : TitaniumCard
        Credit Limit   : 500000
        Annual Charge  : 500

        Card Type      : GoldCard
        Credit Limit   : 1500000
```

```
        Annual Charge  : 1500
* */
```



Flow Diagram



Simple Factory

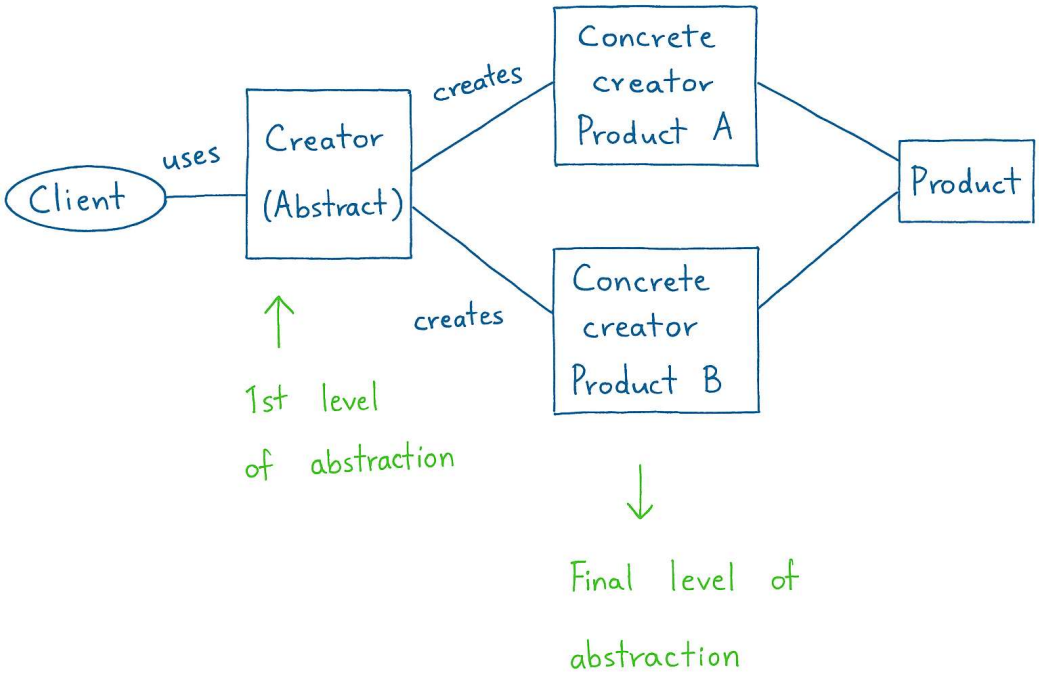
- One class creates different objects.
- A Simple Factory is typically a class with a static method that returns instances of different classes based on input.

Real life examples :

- Simple Factory is like a menu where you tell exactly what you want and the kitchen prepares it.
- It's like a **vending machine**. You press a button, and it gives you the item.

Factory Method

- Subclasses decide what object to create.
- This is a **more flexible** and object-oriented way of creating objects **without knowing the exact class** of the object that will be created.
- Instead of one class deciding what to create, you define an abstract method for creating objects, and **let subclasses decide** what to create.



Feature	Simple Factory	Factory Method
Who creates the object?	One factory class	Each child class
Uses if-else or switch?	Yes	No
Flexible?	Not very flexible	Very flexible

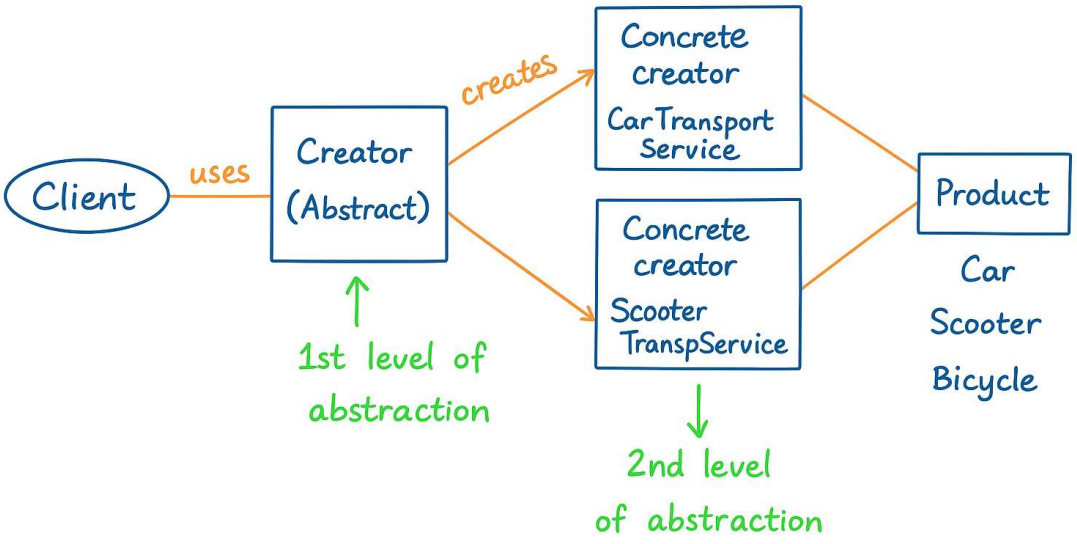
Simple Real-Life Example: Transport Service (Uber)

Let’s say you’re building a transport app like Uber. Depending on the user's choice, you want to book:

- A **Car** for normal users
- A **Scooter** for quick and cheap rides
- A **Bicycle** for eco-friendly rides

Now, how do you create the right transport? You don’t want to write new Car() or new Scooter() everywhere.

Instead, you use the **Factory Method Pattern** to let the system decide **which transport to create**, depending on the context.



How Factory Method Works (Using Uber Example)

1. Create a common interface.

```
public interface Transport {
    void book();
}
```

2. Create child classes - different transport types that implement the interface.

(i) Car

```
public class Car implements Transport {
    @Override
    public void book() {
        System.out.println("Car booked");
    }
}
```

(ii) Scooter

```
public class Scooter implements Transport {
    @Override
    public void book() {
        System.out.println("Scooter booked");
    }
}
```

(iii) Bicycle

```
public class Bicycle implements Transport {
    @Override
    public void book() {
        System.out.println("Bicycle booked");
    }
}
```

3. Create a base factory class with a factory method - Now, instead of using if-else to decide which object to return (like in Simple Factory), we use a **base class** with an **abstract factory method** that will be **implemented by subclasses**.

```
public abstract class TransportService {
    // Factory Method
    protected abstract Transport createTransport();

    // Business Logic - Common for all subclasses
    public void startBooking() {
        Transport transport = createTransport();
```

```
        transport.book();
    }
}
```

4. Subclasses decide what object to create, now we write subclasses that decide which transport to return - Each subclass implements the factory method **createTransport()** and returns the correct transport type.

(i) CarService

```
public class CarService extends TransportService {
    @Override
    protected Transport createTransport() {
        return new Car(); // Creating and returning a Car instance
    }
}
```

(ii) BicycleService

```
public class BicycleService extends TransportService {
    @Override
    protected Transport createTransport() {
        return new Bicycle();
    }
}
```

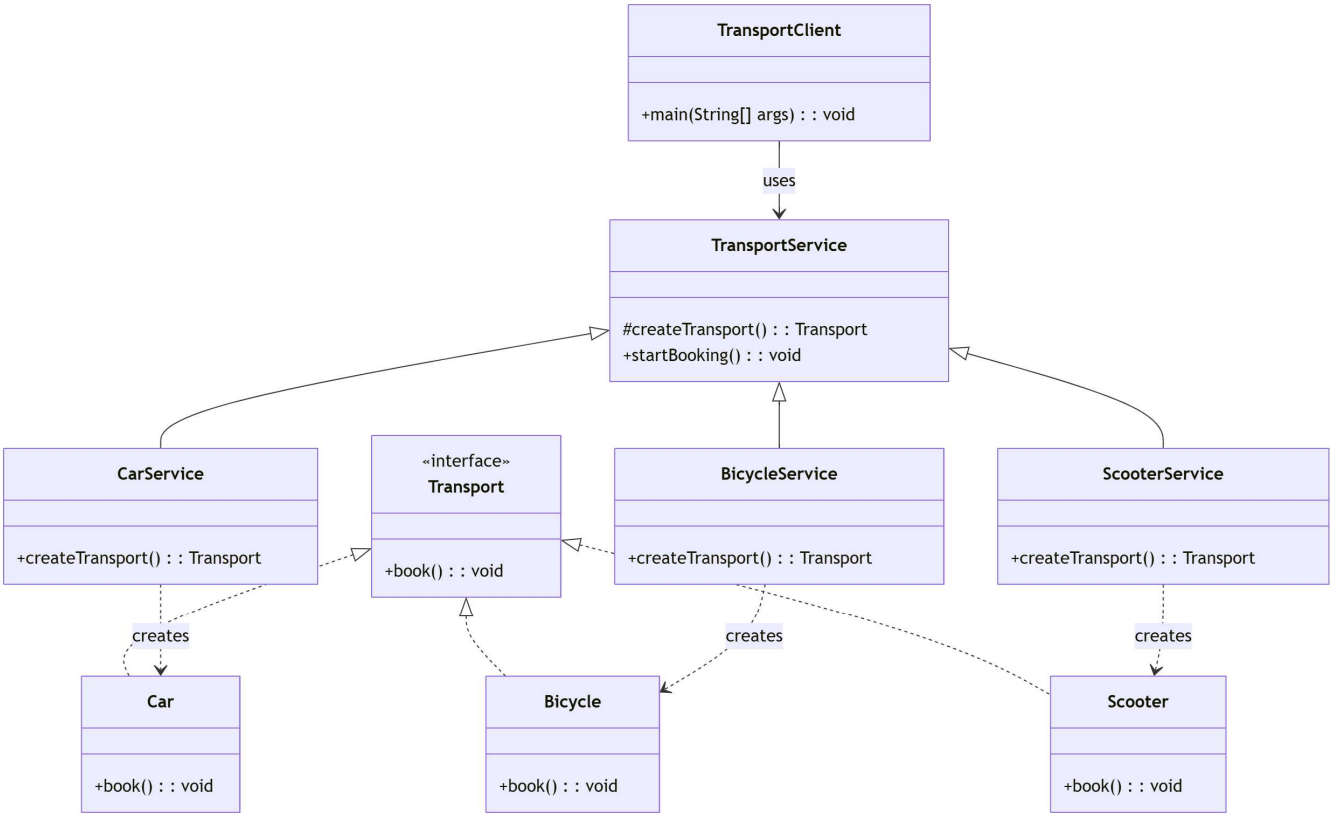
(iii) ScooterService

```
public class ScooterService extends TransportService {
    @Override
```

```
    protected Transport createTransport() {
        return new Scooter();
    }
}
```

5. Client code will only call the factory method, not use new - The client doesn't care **how** or **which object** is created. It just asks the transport service to start booking. It just works with the TransportService and lets the **factory method** do the object creation.

```
public class TransportClient {
    public static void main(String[] args) {
        // Example: User selects "Car" from app
        TransportService carService = new CarService();
        carService.startBooking();  // Car booked

        // Example: User selects "Bicycle" from app
        TransportService bicycleService = new BicycleService();
        bicycleService.startBooking();  // Bicycle booked

        // Example: User selects "Scooter" from app
        TransportService scooterService = new ScooterService();
        scooterService.startBooking();  // Scooter booked
    }
}
```



Another example using restaurant example :

Imagine a **restaurant** with different kitchens:

- ItalianKitchen creates Pizza
- ChineseKitchen creates Noodles
- MexicanKitchen creates Tacos

But the **restaurant staff** doesn't care what food is being made. It just says:

"*Hey Kitchen, cook the meal.*"

Each kitchen (subclass) decides **what to make**. That’s the **factory method**.

1. Create a common interface.

```
public interface Food {
    void prepare();

    void cook();

    void pack();

    String getName();

    int getPreparationTime();
}
```

2. Create child classes - different food types that implement the interface.

(i) Pizza

```
public class Pizza implements Food {
    @Override
    public void prepare() {
        System.out.println("Preparing pizza dough, cheese and
toppings.");
    }

    @Override
    public void cook() {
        System.out.println("Baking pizza in oven.");
    }

    @Override
    public void pack() {
```

```
        System.out.println("Packing pizza in pizza box.");
    }

    @Override
    public String getName() {
        return "Italian Pizza";
    }

    @Override
    public int getPreparationTime() {
        return 20;
    }
}
```

(ii) Noodles

```
public class Noodles implements Food {
    @Override
    public void prepare() {
        System.out.println("Chopping vegetables and boiling
noodles.");
    }

    @Override
    public void cook() {
        System.out.println("Stir frying noodles.");
    }

    @Override
    public void pack() {
        System.out.println("Packing noodles in takeaway box.");
    }

    @Override
    public String getName() {
```

```
        return "Chinese Noodles";
    }

    @Override
    public int getPreparationTime() {
        return 15;
    }
}
```

(iii) Tacos

```
public class Tacos implements Food {
    @Override
    public void prepare() {
        System.out.println("Preparing taco shells and fillings.");
    }

    @Override
    public void cook() {
        System.out.println("Grilling taco filling.");
    }

    @Override
    public void pack() {
        System.out.println("Packing tacos carefully.");
    }

    @Override
    public String getName() {
        return "Mexican Tacos";
    }

    @Override
    public int getPreparationTime() {
        return 18;
```

```
    }
}
```

3. Create a base factory class with a factory method.

```
public abstract class KitchenService {
    // Factory Method
    protected abstract Food cookMeal();

    // Common Business Logic
    public final void processOrder() {
        System.out.println("Customer placed an order.");

        Food food = cookMeal();

        System.out.println("Selected Food : " + food.getName());

        System.out.println("Estimated Time : " +
food.getPreparationTime() + " minutes");

        food.prepare();

        food.cook();

        food.pack();

        generateBill(food);

        System.out.println("Order Ready!");

        System.out.println("--------------------------------");
    }
```

```
    private void generateBill(Food food) {
        System.out.println("Generating bill for " + food.getName());
    }
}
```

4. Subclasses decide what object to create, now we write subclasses that decide which food to return.

(i) ItalianKitchen

```
public class ItalianKitchen extends KitchenService {
    @Override
    protected Food cookMeal() {
        return new Pizza();
    }
}
```

(ii) ChineseKitchen

```
public class ChineseKitchen extends KitchenService {
    @Override
    protected Food cookMeal() {
        return new Noodles();
    }
}
```

(iii) MexicanKitchen

```
public class MexicanKitchen extends KitchenService {
    @Override
    protected Food cookMeal() {
        return new Tacos();
    }
```

```
}
```

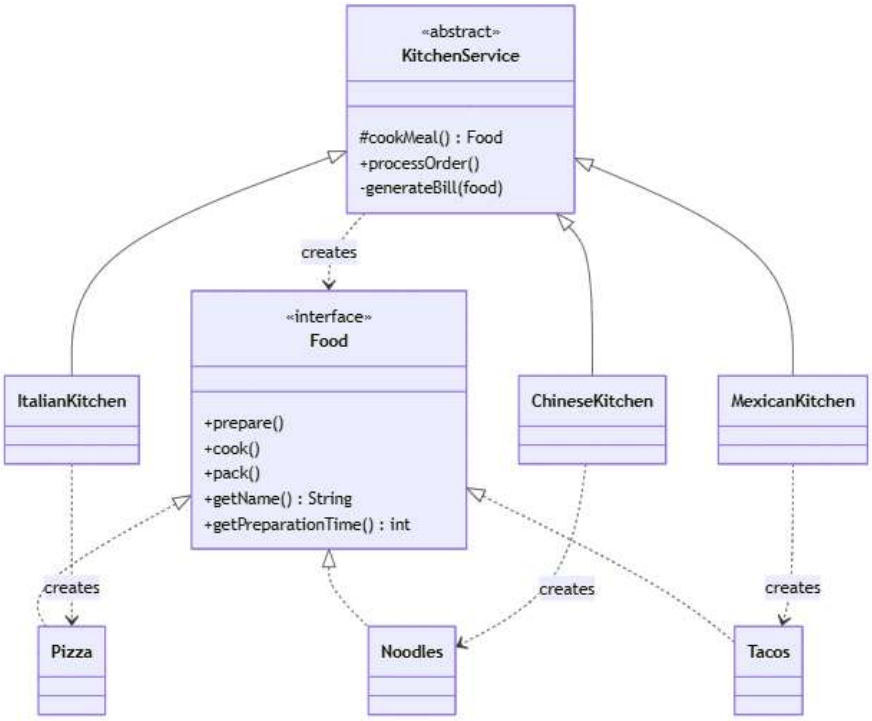
5. Client code will only call the factory method.

```
public class Main {
    public static void main(String[] args) {
        KitchenService italian = new ItalianKitchen();
        italian.processOrder();

        KitchenService chinese = new ChineseKitchen();
        chinese.processOrder();

        KitchenService mexican = new MexicanKitchen();
        mexican.processOrder();
    }
}
```

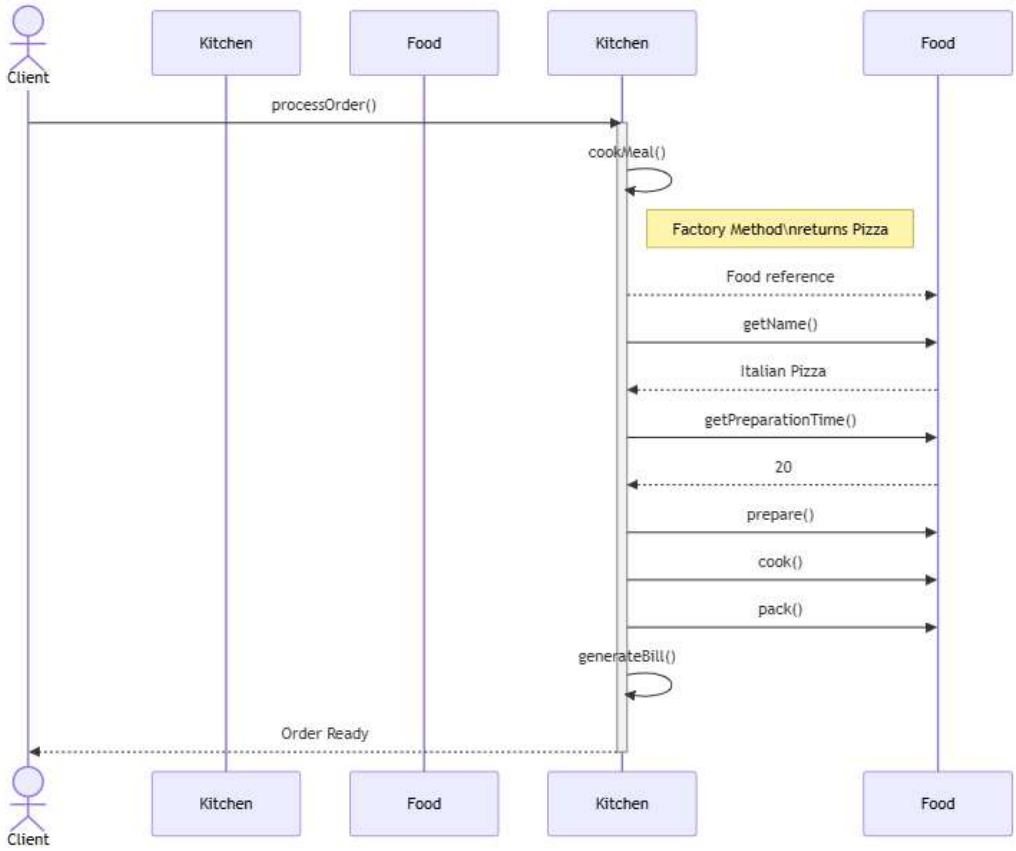
```
/* *
    Output :
        Customer placed an order.
        Selected Food : Italian Pizza
        Estimated Time : 20 minutes
        Preparing pizza dough, cheese and toppings.
        Baking pizza in oven.
        Packing pizza in pizza box.
        Generating bill for Italian Pizza
        Order Ready!
        ---------------------------------
        Customer placed an order.
        Selected Food : Chinese Noodles
        Estimated Time : 15 minutes
        Chopping vegetables and boiling noodles.
        Stir frying noodles.
```

```
Packing noodles in takeaway box.
Generating bill for Chinese Noodles
Order Ready!
---------------------------------
Customer placed an order.
Selected Food : Mexican Tacos
Estimated Time : 18 minutes
Preparing taco shells and fillings.
Grilling taco filling.
Packing tacos carefully.
Generating bill for Mexican Tacos
Order Ready!
---------------------------------
* */
```

Class Diagram



Sequence Diagram



Why Use Factory Method?

- **Easy to extend -** Just add a new subclass for new product (e.g., XUV).
- **No need to change core logic -**Your main code stays the same.
- **Cleaner code -** No if-else or switch to check types.
- **Follows OOP principles -** Like Open/Closed Principle (open for

extension, closed for modification).